

Projet 9

Améliorez l'organisation Salesforce du client Fasha

Livrable n°4

Documentation technique : tests, performances et CI/CD

Parcours Développeur Salesforce - OpenClassrooms

Abderrahim Yachkour

Développeur Salesforce junior - Cabinet SFQUAL

Client : Fasha (distributeur de vêtements)

Date de remise : Juin 2026

Sommaire

1. Tests et couverture de code	3
1.1. Objectif	3
1.2. Stratégie en deux niveaux	3
1.3. Classes de tests Apex	3
1.4. Comment lancer les tests	3
2. Performances.....	4
2.1. Les trois limites gouverneurs critiques	4
2.2. Scénario clé : activation de 200 commandes	4
2.3. NetAmount en champ Formule	4
2.4. Synthèse des gains mesurés	4
2.5. Lien avec l'efficacité énergétique	5
3. Pipeline CI/CD	5
3.1. Architecture en un coup d'œil	5
3.2. Secrets GitHub requis	5
3.3. Étapes du workflow	6
3.4. Pourquoi le calcul du delta est important	6
3.5. Comment maintenir le pipeline	6
4. Conclusion.....	7

Ce document regroupe en une vue synthétique les trois piliers techniques du projet Fasha : la stratégie de tests et la couverture de code, les optimisations de performance, et le pipeline CI/CD GitHub Actions. Il complète le Livrable 1 (Rapport de tests) et le Livrable 5 (Rapport de performance détaillé) qui apportent le détail des mesures.

1. Tests et couverture de code

1.1. Objectif

Salesforce impose 75 % de couverture Apex minimum pour autoriser un déploiement en production. Sur ce projet, la couverture mesurée atteint 92 % (détail dans le Livrable 1) : chaque cas métier critique est couvert par un test dédié, pas seulement les chemins faciles.

1.2. Stratégie en deux niveaux

- Tests Apex (serveur) : une classe de test par classe métier (Service, Selector, Trigger Handler, Controller, Batch). Isoler par couche permet d'identifier instantanément la responsabilité d'un échec.
- Tests Jest (LWC) : le composant orders est testé avec le framework Jest officiel Salesforce, en simulant les retours du wire Apex (succès, erreur, valeur vide).

1.3. Classes de tests Apex

- AccountCAServiceTest : recalcul correct, robustesse aux Sets vides ou null, exclusion des statuts Draft et Cancelled.
- OrderSelectorTest : retour vide sur Set vide, filtrage par statut, agrégation SUM groupée par compte.
- OrderTriggerHandlerTest : insertion, transitions de statut, suppression, modification triviale (pas de double comptage), changement de compte (recalcul de l'ancien ET du nouveau), bulk 200 commandes (zéro exception gouverneur).
- MyTeamOrdersControllerTest : retour zéro sur accountId null, somme correcte sur compte avec commandes Activated, comportement attendu sur compte sans commande Activated.
- UpdateAllAccountsTest : exécution complète du batch, vérification du recalcul sur l'ensemble des comptes éligibles.

1.4. Comment lancer les tests

- Tests Apex en local : `sf apex run test --test-level RunLocalTests --code-coverage --result-format human --wait 10`
- Tests Jest LWC : `npm test`
- Tests Apex en CI : exécutés automatiquement par le workflow `main_deploy.yml` lors de chaque pull request vers main.

Overall Code Coverage 		
Class	Percent	Lines
Overall	92%	
AccountCAService	100%	13/13
MyTeamOrdersController	100%	8/8
OrderSelector	100%	11/11
OrderTrigger	100%	1/1
OrderTriggerHandler	86%	33/38
UpdateAllAccounts	90%	9/10

2. Performances

2.1. Les trois limites gouverneurs critiques

- 100 requêtes SOQL maximum par transaction.
- 150 opérations DML maximum par transaction.
- 10 secondes de CPU time maximum.

Tout dépassement déclenche une System.LimitException et annule la transaction entière. Le respect de ces limites conditionne donc le bon fonctionnement de l'application.

2.2. Scénario clé : activation de 200 commandes

Avant correction

Le trigger UpdateAccountCA exécutait une SOQL et un DML à l'intérieur d'une boucle for. Pour 200 commandes : 200 SOQL et 200 DML, soit le double de la limite SOQL. L'exception Too many SOQL queries: 101 était levée systématiquement, opération annulée.

Après correction

L'architecture en trois couches (Trigger Handler, Service, Selector) consomme une seule SOQL agrégée GROUP BY et une seule opération DML groupée, quel que soit le volume. La consommation est constante.

2.3. NetAmount en champ Formule

Le trigger CalculMontant a été supprimé et remplacé par un champ Formule (TotalAmount - ShipmentCost__c). Coût d'écriture : zéro. Le calcul est effectué à la lecture par le moteur Salesforce, qui peut le mettre en cache. Bénéfice supplémentaire : un bug d'origine (seule la première commande du lot était calculée) est éliminé par construction.

2.4. Synthèse des gains mesurés

200 commandes activées simultanément

- Avant : exception dès la 101ème commande, opération annulée.
- Après : 1 SOQL, 4 DML, 341 ms de CPU (sur 10 000 ms autorisés), soit moins de 3,5 % du budget.

Calcul du NetAmount sur 200 commandes

- Avant : 0,5 % des commandes correctement calculées, 200 DML inutiles.
- Après : 100 % des commandes à jour, 0 DML (calcul à la lecture).

2.5. Lien avec l'efficacité énergétique

Chaque SOQL et chaque DML économisés réduisent directement le CPU, les entrées-sorties disque et le logging sur les datacenters Salesforce. Les leviers actionnés (suppression des SOQL en boucle, agrégation GROUP BY, mutualisation via le pattern Selector, champ Formule à la place d'un trigger) sont cohérents avec une démarche de numérique responsable.

Log executeAnonymous @6/23/2026, 9:57:08 PM		
Execution Log		
Timestamp	Event	Details
21:57:09:102	LIMIT_USAGE_FOR_NS	(default)
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of SOQL queries: 1 out of 100
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of query rows: 1 out of 50000
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of SOSL queries: 0 out of 20
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of DML statements: 4 out of 150
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of Publish Immediate DML: 0 out of 150
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of DML rows: 203 out of 10000
21:57:08:000	LIMIT_USAGE_FOR_NS	Maximum CPU time: 341 out of 10000
21:57:08:000	LIMIT_USAGE_FOR_NS	Maximum heap size: 0 out of 6000000
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of callouts: 0 out of 100
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of Email Invocations: 0 out of 10
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of future calls: 0 out of 50
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of queueable jobs added to the queue: 0 out of 50
21:57:08:000	LIMIT_USAGE_FOR_NS	Number of Mobile Apex push calls: 0 out of 10
21:57:08:000	LIMIT_USAGE_FOR_NS	

3. Pipeline CI/CD

3.1. Architecture en un coup d'œil

- Un workflow unique : .github/workflows/main_deploy.yml.
- Déclencheurs : pull request vers main (validation dry-run) et push sur main (déploiement réel).
- Outils : Salesforce CLI (sf) pour authentification JWT et déploiement, sfdx-git-delta pour calculer uniquement les métadonnées modifiées.

3.2. Secrets GitHub requis

- SF_CONSUMER_KEY : Consumer Key de la Connected App Salesforce.
- SF_USERNAME : email du user Salesforce utilisé pour le déploiement.
- SF_JWT_KEY : contenu de la clé privée RSA (server.key, encodée en base64).
- SF_INSTANCE_URL : https://login.salesforce.com (prod) ou https://test.salesforce.com (sandbox).

Configuration : Settings, Secrets and variables, Actions, New repository secret.

3.3. Étapes du workflow

- Checkout du repo avec fetch-depth 0 (pour permettre à sfdx-git-delta de comparer les commits).
- Installation de Node.js 20, du Salesforce CLI et de sfdx-git-delta.
- Authentification JWT à l'org cible à partir des secrets.
- Génération du delta : sfdx source delta compare origin/main (PR) ou HEAD~1 (push) à HEAD, et produit un dossier delta/force-app contenant uniquement les fichiers modifiés.
- Sur PR : sf project deploy validate avec test-level RunLocalTests (dry-run).
- Sur push to main : sf project deploy start avec test-level RunLocalTests (déploiement réel).
- Cleanup : suppression de la clé privée locale.

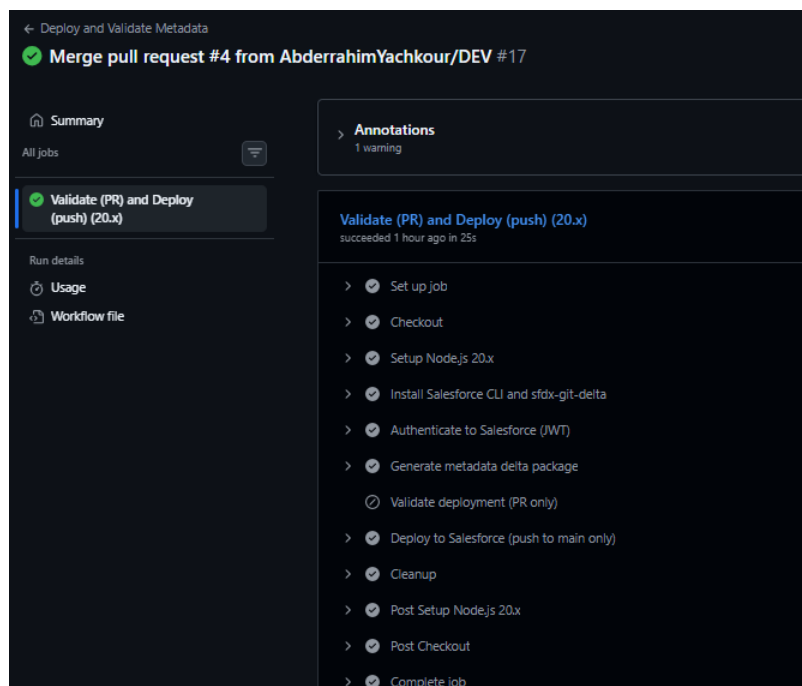
3.4. Pourquoi le calcul du delta est important

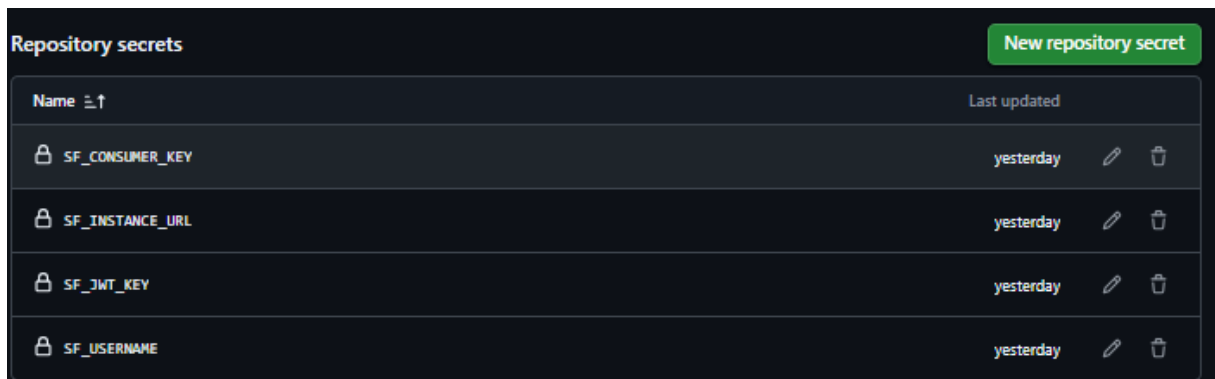
Sans delta, chaque commit redéploierait l'intégralité du dossier force-app. Sur un projet réel cela représente des centaines de fichiers et plusieurs minutes de tests inutiles. Avec sfdx-git-delta, on ne déploie que ce qui a changé : pipeline rapide, factures Salesforce limitées, et historique de déploiement lisible.

3.5. Comment maintenir le pipeline

- Si un secret expire (typiquement la clé JWT) : régénérer le certificat, mettre à jour la Connected App côté Salesforce, et remplacer le secret SF_JWT_KEY côté GitHub.
- Pour ajouter une nouvelle vérification (ex : analyse statique PMD) : insérer une étape supplémentaire avant le step de validation.
- Pour cibler une sandbox au lieu de la prod : changer la valeur de SF_INSTANCE_URL en <https://test.salesforce.com>.

L'onglet Actions du dépôt GitHub montrant l'historique des runs



Settings, Secrets and variables, Actions, montrant les 4 secrets configurés (valeurs masquées)

The screenshot shows the 'Repository secrets' interface in GitHub. At the top right is a green button labeled 'New repository secret'. Below it is a table with two columns: 'Name' and 'Last updated'. There are four rows, each representing a secret. Each row has a lock icon, the secret name, the last updated time (all are 'yesterday'), and edit/delete icons.

Name	Last updated
SF_CONSUMER_KEY	yesterday
SF_INSTANCE_URL	yesterday
SF_JWT_KEY	yesterday
SF_USERNAME	yesterday

4. Conclusion

Les tests garantissent que les corrections fonctionnent dans tous les cas métier critiques, y compris en bulk. Les optimisations de performance ramènent la consommation de ressources Salesforce bien en dessous des limites gouverneurs, ce qui permettra à Fasha de tenir la charge à mesure que le volume de commandes augmentera. Le pipeline CI/CD garantit qu'aucun code non testé ou non validé n'arrive en production : chaque pull request est validée par dry-run, chaque merge sur main déclenche un déploiement automatisé et tracé.